Big Data and Machine Learning for Applied Economics

Week 07: Representing: Compressing with Purpose

Eduard F. Martinez Gonzalez

Department of Economics, Universidad Icesi

October 12, 2026

Previously...

- **We opened the boosting's box:** the department first, ethnicity next; the interactions the machine found are, above all, ethnicity within each department (Afro $\times$ department: 2.4 points).
- **The clue:** permuting a census block raised the error between 1.3 and 3.8 points; retraining without it, only 0.3 to 0.4. The blocks stand in for one another: the 37 columns tell few stories, repeated.
- **And where it fails:** 14.9 error in rural areas against 9.3 in town centers. With the map, another explanation with almost the same error (12.1).

What was left pending

How many distinct things does the census say about a station? If they are few, they should be summarizable in a few indices or a few territory types. And who should decide how to summarize them?

Problem Set 1 debrief: the ranking

Fifteen minutes [table and comments to be filled in after grading]

1. **The standings:** each team's test RMSE for its final model, next to its validation RMSE. Do they match? A team whose test came in far above its validation learned something today.
2. **What the winner did differently:** which model it chose, which variables it built inside the fold, and which validation scheme it used.
3. **The frequent stumbles:** the test set opened more than once; the recipe (imputing, scaling, grouping) fit outside the fold; random validation with listings from the same zone in training and validation.
4. **The course's number:** how much regularization gained over OLS with all the variables, and how much the error moved when validating by zones instead of at random. [to be filled in]

What comes next

Problem Set 2 starts from Problem Set 1's final model: the course ranking is recomputed on November 24 with the final model of both.

Today's question: how many distinct things does the census know about a station?

And who decides how to summarize them: the variance of the x's or the y

- **Without y, in straight lines:** how many directions concentrate the information in the 37 columns? Principal components (PCA).
- **Without y, in groups:** how many territory types are there? k-means. Nobody looks at the vote when building them: only the structure of the x's.
- **With y:** what if the summary is learned *in order to* predict? A **neural network** manufactures its own variables and fits them together with the prediction. The exam, the usual one: the test set against boosting.
- **Without columns:** when the data point is an image, there is no sheet of variables to summarize: the representation is learned, or borrowed.

The takeaway

To represent is to compress with purpose. Without a y, the purpose comes from the variance of the x's; with a y, from the prediction. What varies most is not, necessarily, what predicts.

1. Without y, in straight lines: principal components
2. Without y, in groups: k-means
3. With y: the neural network
4. How a network is fitted
5. When there are no columns: networks that see
6. Closing

What changes when there is no y

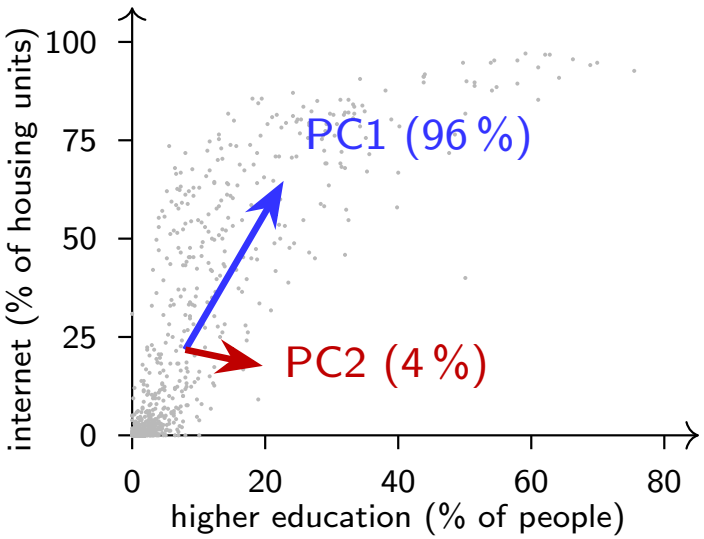
	Supervised (sessions 2 to 6)	Unsupervised (today, the first hour)
Goal	predict y	describe the structure of the x's
Criterion	test error	there is no right answer
Validation	CV and a test set	stability, interpretation, downstream use
Danger	overfitting	**seeing patterns where there are none**

How do you validate, then? With three imperfect substitutes for the test error:

1. **Stability:** do the groups or components survive another seed, another sample, dropping a variable?
2. **Interpretation:** does the structure have an economic reading you can name?
3. **Downstream use:** if the index or group later *predicts* something (consumption, default, the vote), that task audits it. That is how we will audit ours: with the vote, which they never saw.

The direction that concentrates the variance

ISL 12.2.1. Two census columns at 800 training stations: correlation 0.82



The first component is the linear combination of the centered columns with the largest variance:

$$z_{i1} = \sum_{j=1}^{p} \phi_{j1}\, x_{ij},$$

$$\max_{\phi_1} \frac{1}{n} \sum_{i=1}^{n} z_{i1}^2 \quad \text{s.a.} \quad \sum_{j=1}^{p} \phi_{j1}^2 = 1.$$

- The ϕ's are the **loadings**; the z's, the **scores** (each station's projection).
- **PC2**: the largest-variance one among those perpendicular to PC1. It is an eigenvalue problem (appendix A1).
- Here PC1 takes 96 %; without standardizing, internet, which varies more, tilts it toward its own axis.

The other face: the closest approximation

ISL 12.2.2 and 12.2.3: maximizing the variance is the same as minimizing the approximation error

The first M components give the best M-dimensional approximation to the data:

$$x_{ij} \approx \sum_{m=1}^{M} z_{im}\,\phi_{jm},$$

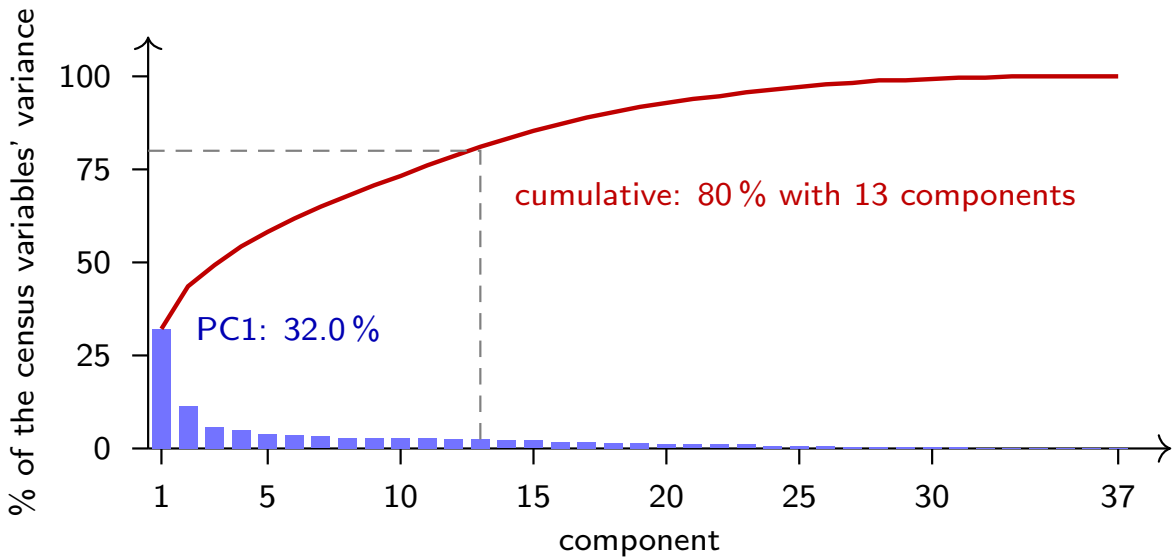
y la varianza total se parte en dos pedazos que suman siempre lo mismo:

$$\underbrace{\sum_j \frac{1}{n}\sum_i x_{ij}^2}_{\text{variance of the data}} = \underbrace{\sum_{m=1}^{M} \frac{1}{n}\sum_i z_{im}^2}_{\text{variance of the } M \text{ components}} + \underbrace{\frac{1}{n}\sum_j\sum_i \left(x_{ij} - \sum_{m=1}^{M} z_{im}\phi_{jm}\right)^2}_{\text{approximation error}}.$$

- That is why the **proportion of variance explained** (PVE) is an R^2: that of approximating the x's with M components, $1 - \mathrm{RSS}/\mathrm{TSS}$.
- **Standardizing:** with columns on different scales, the largest-variance one steals PC1 (ISL 12.2.4). Our 37 are standardized with the **training** means and deviations: even without a y there is a pipeline.

How many components? The variance of our 37 columns

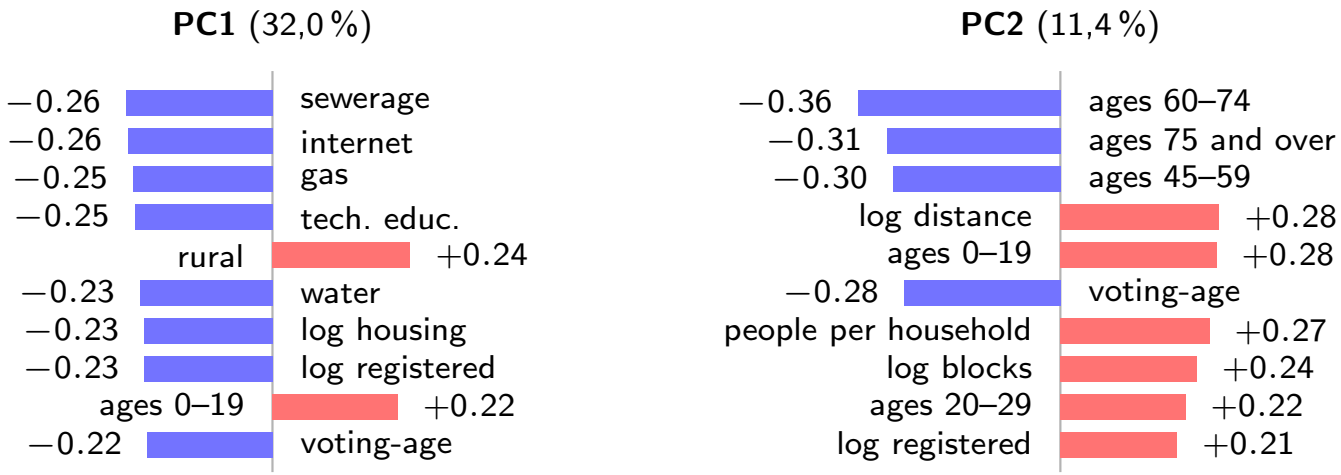
The standardized census variables of the 9,602 training stations



- **One direction concentrates nearly a third** (32.0 %) of the variance of 37 columns; the second, 11.4 %. Reaching 80 % takes 13 components; 90 %, 18. It is week 6's redundancy, measured.
- **The last one explains exactly zero:** `edad_votar` + `edad_0_19` = 100 at every station. There are 37 columns and 36 dimensions; PCA exposes the repeated column.
- **No criterion is automatic:** the elbow, a cumulative threshold, or interpretation. If the destination is prediction, M is chosen by the judge (later today).

What each component says: the loadings

The ten largest loadings of PC1 and PC2; a component's sign is arbitrary

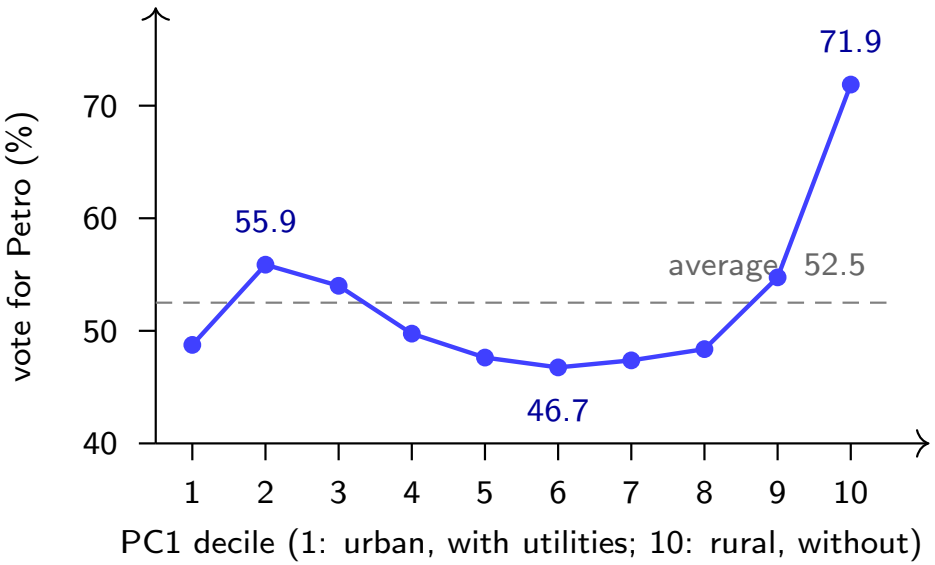


- **PC1 is a deprivation index:** utilities, technical education, and the station's size on one side; rurality and children on the other. Correlation with rurality 0.84; with higher education, −0.75.
- **PC2 separates young, remote stations from aging ones:** correlation 0.57 with the population aged 0 to 19. Nobody told PCA what age or distance is: it found them because they vary together.

Question for the room: PC1 summarizes nearly a third of the census's variance. How much of the Petro vote does it explain?

The index built without y, audited with y

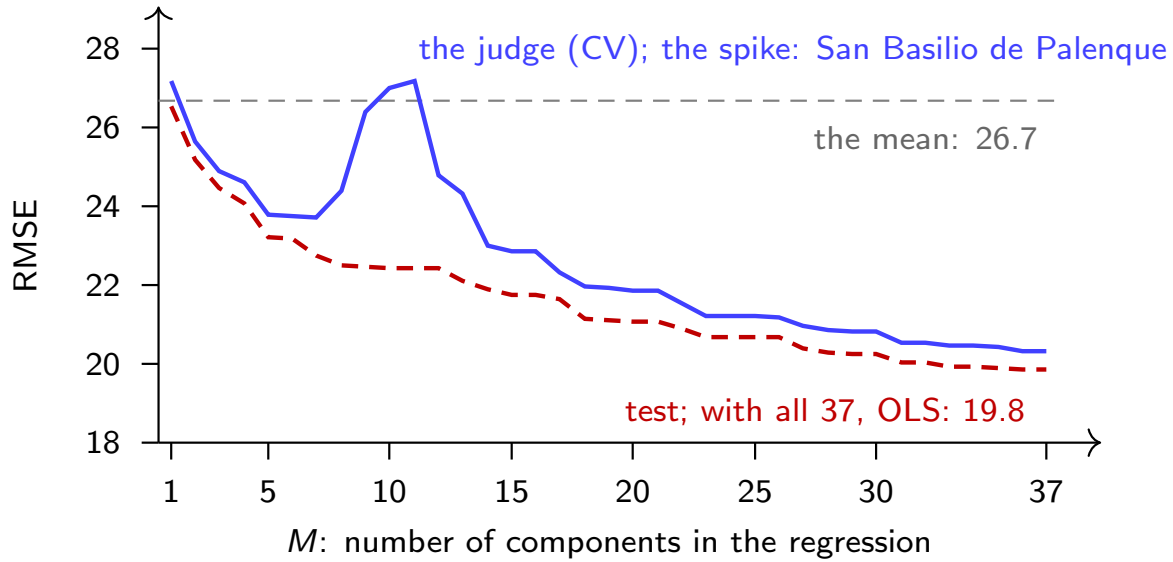
The Petro vote in each decile of PC1 (9,602 training stations)



- **The correlation with the vote is 0.09:** the direction that varies most explains 0.9 % of the vote. But not in a straight line: it rises in decile 2 (55.9), falls in the middle (46.7), and shoots up in the most deprived (71.9), which is also the most Afro (33 % against 2 % in the first): the Pacific.
- **The asset index** (Filmer and Pritchett, 2001) is this very object: the PC1 of what the household owns is its socioeconomic level. It is the DHS surveys' wealth index, and the y that Jean et al. (2016) predict with satellite images (part 5).

Compressing to predict, without looking at y: principal components regression

PCR: OLS on the scores of the first M components; the PCA is estimated inside each fold



- **Compressing blindly costs:** with one component, 26.5 (the mean: 26.7); with five, 23.2; with all of them, 19.8, which is OLS. The judge picks 36: every one with variance. With 9,602 stations and 37 columns there is no variance to save.
- **The judge's spike has a name:** PC10 is the `palenquero` column alone (loading 0.98), zero at 89 % of the stations. Standardized, San Basilio de Palenque is an entire direction; out of fold, the PCR predicts 1,465 for it (it voted 95).

1 Without y, in straight lines: principal components

2 Without y, in groups: k-means

3 With y: the neural network

4 How a network is fitted

5 When there are no columns: networks that see

6 Closing

k-means: the problem and the algorithm

ISL 12.4.1: split the stations into K groups as alike inside as possible

The problem: choose the groups $C_1, \ldots, C_K$ that minimize the within-group variation,

$$\min_{C_1,\ldots,C_K} \sum_{k=1}^{K} \frac{1}{|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2 \;=\; \min_{C_1,\ldots,C_K} \sum_{k=1}^{K} 2 \sum_{i \in C_k} \sum_{j=1}^{p} (x_{ij} - \bar{x}_{kj})^2.$$

The identity (ISL 12.18) says that the distances between pairs in the same group are the distances to the **centroid** $\bar{x}_k$, the group's mean.

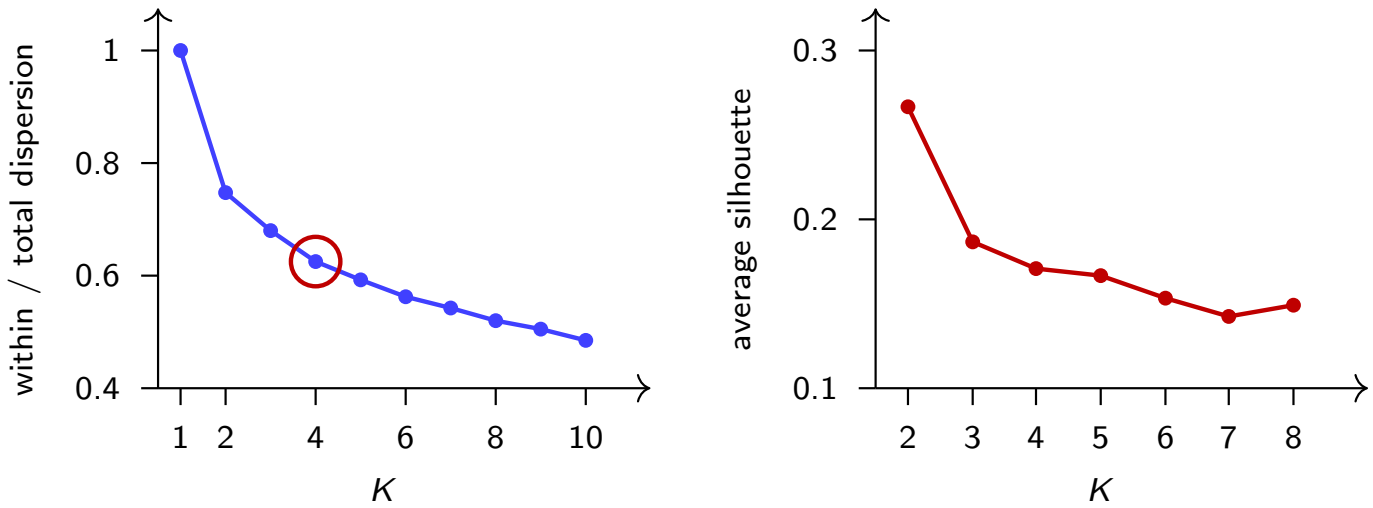
The algorithm (there are nearly K^n partitions: you cannot try them all):

1. Assign each station a group at random.
2. Repeat until nothing changes: (a) compute each group's centroid; (b) reassign each station to the nearest centroid.

- Each step lowers the objective, so it converges… to a **local** optimum: it is repeated with 25 starts and the best one stays.
- Like everything that uses distances, with standardized columns (week 2's k-NN).

How many groups?

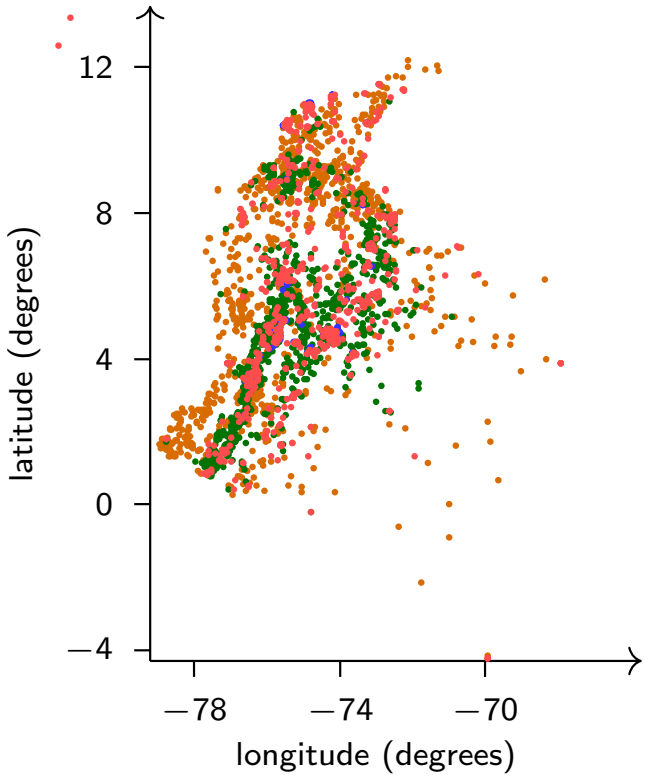
Left: within-group variation over the total. Right: the silhouette on 2,000 random stations



- **The within variation always falls with K:** from 1 to 0.75 with two groups, 0.62 with four, 0.48 with ten. The elbow is soft: no kink decides on its own.
- **The silhouette** compares, for each station, its mean distance to its own group with that to the neighboring group (from -1 to 1). It prefers two groups (0.27); with four, 0.17.
- **K is a decision, not a true parameter:** if a campaign can design four territorial strategies, $K = 4$.

Four Colombias: the census typologies

k-means with $K = 4$ on the standardized census variables; the vote, only to audit



	Stations	Rural	Educ.	Ethnicity	Vote
ethnic rural	2,721	96 %	1.9	48.6	**63.0**
Andean rural	2,807	96 %	2.6	8.6	**42.2**
serviced urban	1,388	1 %	29.3	2.6	49.8
low-income urban	2,686	10 %	8.8	8.4	54.0

Educ.: higher education (% of people). Ethnicity: Afro + Indigenous (%). Vote: for Petro (%).

- **They let themselves be named:** two urban Colombias split by services, and two ruralities split by ethnicity.
- **The map gives them away:** *k*-means never saw the coordinates, and the Pacific and the borders come out one color, the Andean axis another.
- **Rurality is not one thing:** the ethnic one votes 63.0 and the Andean one 42.2, 21 points apart. That is why week 6's rural error is large.

Do the groups help predict?

Describing is not predicting: the test vote with each group's mean

- **With the group's mean:** 25.6 RMSE (the country's mean: 26.7). With the department's, 18.0; with the group's within each department, 16.1. Four territory types are not enough to predict a station; they serve to *talk* about it.
- **Stability:** with five other seeds, 100 % of the stations stay in the same group. These groups do not dance.
- **With** $K = 5$ the ethnic rurality splits and a core of 867 stations appears (Afro 36 %, Indigenous 41 %) voting 78.2.

Clustering's warnings

Scale rules (standardize); k-means looks for round, same-sized groups; one outlier can hijack a centroid; and the algorithm **always** delivers K groups, even on pure noise. A group is a finding when it survives stability, interpretation, and a y it never saw.

1 Without y, in straight lines: principal components

2 Without y, in groups: k-means

3 With y: the neural network

4 How a network is fitted

5 When there are no columns: networks that see

6 Closing

From PCR to the network: the same formula

ISL 10.1: a one-layer network is a regression on variables it manufactures itself

PCR regresses y on M directions chosen by the variance of the x's:

$$f(x) \;=\; \beta_0 + \sum_{m=1}^{M} \beta_m \underbrace{\phi_m' x}_{\text{PC } m}.$$

The neural network regresses y on K directions chosen by the loss on y, bent by a function g:

$$f(x) \;=\; \beta_0 + \sum_{k=1}^{K} \beta_k \underbrace{g\left(w_{k0} + w_k' x\right)}_{A_k:\ \text{unit } k}.$$

The **activations** A_k are derived variables, like week 2's spline bases, but **learned**.
Two differences with PCR: who chooses the directions (the y, not the variance) and whether they are bent (g).

Question for the room: does a single-unit network beat the bent PC1? And OLS with the 37 columns?

From PCR to the network: the same formula

ISL 10.1: a one-layer network is a regression on variables it manufactures itself

PCR regresses y on M directions chosen by the variance of the x's:

$$f(x) \;=\; \beta_0 + \sum_{m=1}^{M} \beta_m \underbrace{\phi_m' x}_{\text{PC } m}.$$

The neural network regresses y on K directions chosen by the loss on y, bent by a function g:

$$f(x) \;=\; \beta_0 + \sum_{k=1}^{K} \beta_k \underbrace{g\left(w_{k0} + w_k' x\right)}_{A_k:\ \text{unit } k}.$$

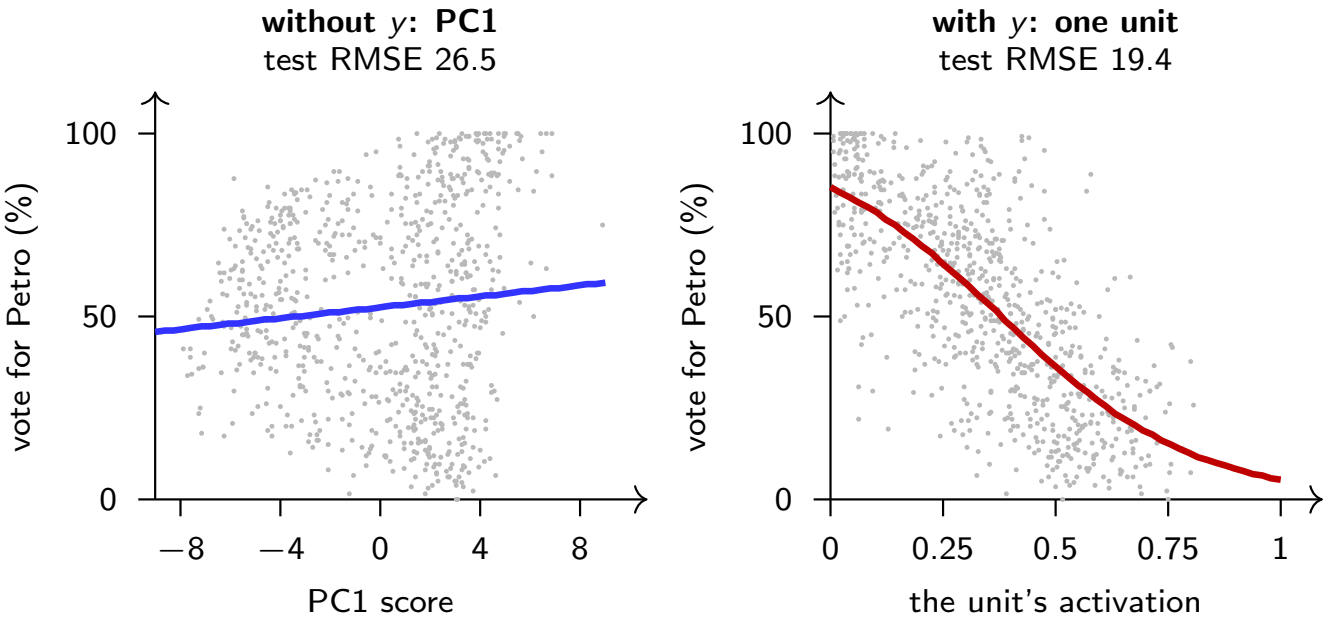
The **activations** A_k are derived variables, like week 2's spline bases, but **learned**.
Two differences with PCR: who chooses the directions (the y, not the variance) and whether they are bent (g).

Question for the room: does a single-unit network beat the bent PC1? And OLS with the 37 columns?

The PC1, by a lot (19.4 against 25.8); OLS, barely (19.4 against 19.8). What matters is who chose the direction.

A single unit: who chooses the direction?

The vote of 800 test stations against PC1 (left) and against the unit of a $K = 1$ network (right)



- **Choosing the direction with y is worth more than bending it:** PC1 in a straight line leaves 26.5 RMSE and bent with a spline, 25.8; the single-unit network, 19.4 (OLS with the 37: 19.8). Its direction barely resembles PC1 (correlation −0.20).
- **The network manufactured a variable:** its two largest weights are the log of population (5.8) and of housing (−5.9), nearly opposite: their difference is people per housing unit.

Why the curve g is needed

ISL 10.1: without g, stacking units adds nothing; with g, curves and interactions appear

The g is week 4's sigmoid, $1/(1+e^{-z})$, or ReLU, $\max(0,z)$; in our network the output also passes through a sigmoid to stay between 0 and 100. **Without** g, the network is linear no matter how many units it has: $\beta_0 + \sum_k \beta_k(w_{k0} + w_k'x)$ is another line in x, OLS's.

With g**, two units already manufacture an interaction** (ISL 10.6 to 10.8). With $p = 2$, $g(z) = z^2$, $\beta_1 = \frac{1}{4}$, $\beta_2 = -\frac{1}{4}$, and

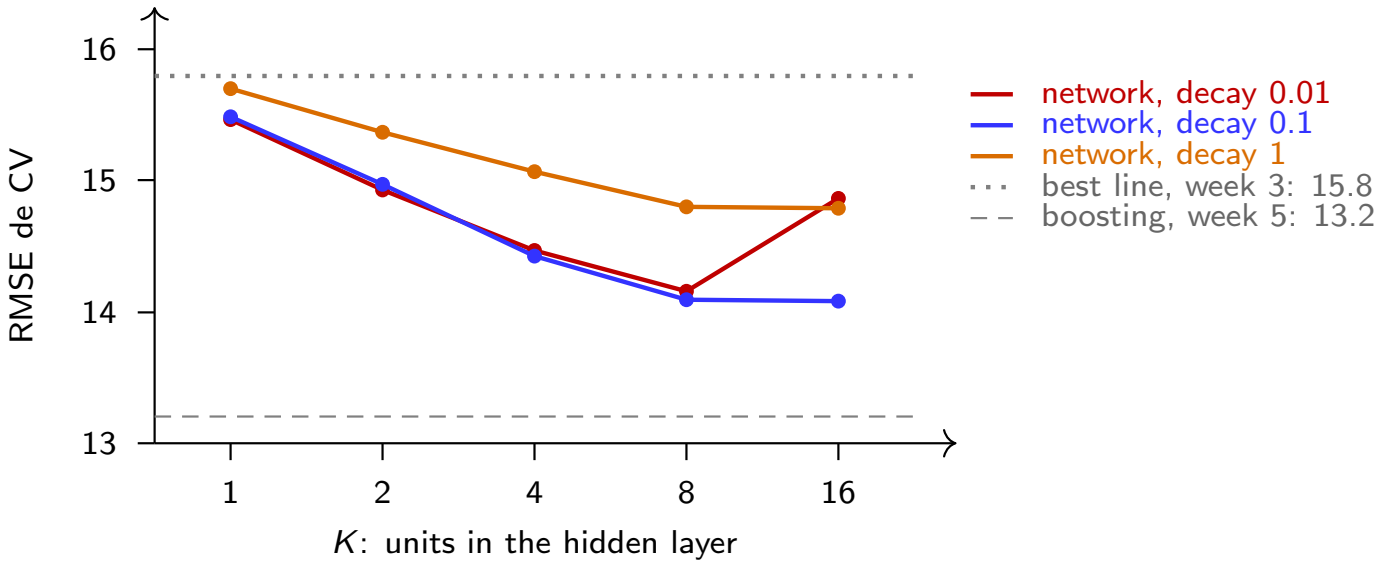
$$A_1 = (x_1 + x_2)^2, \qquad A_2 = (x_1 - x_2)^2,$$

$$f(x) = \tfrac{1}{4}\left[(x_1 + x_2)^2 - (x_1 - x_2)^2\right] = x_1 x_2.$$

- It is the kind of interaction week 3 left unwritten and that in week 5 the trees found with one split inside another. The network finds it by combining bent directions.
- With sigmoids or ReLU you do not get exactly $x_1 x_2$, but you do get any smooth shape with enough units (appendix A6). **The price:** with many units, the network also learns the noise.

More units, more capacity: the U, again

Week 3's judge (ten folds) for $K = 1, 2, 4, 8, 16$ units; 270 networks fitted



- **With the census and the department** the judge picks $K = 16$ and decay 0.1: 14.1 of CV, against 13.2 for boosting with the same ingredients. With the census alone, 17.5 against 15.8.
- **Weight decay is Ridge** (week 3): with 16 units and little penalty (0.01) the network overfits (14.9); with 0.1, it does not. That is 1,153 weights for 9,602 stations.

1 Without y, in straight lines: principal components

2 Without y, in groups: k-means

3 With y: the neural network

4 How a network is fitted

5 When there are no columns: networks that see

6 Closing

The loss is not convex: going down the gradient

ISL 10.7

The problem is nonlinear least squares,

$$\min_{\theta} \; R(\theta) = \frac{1}{2} \sum_{i=1}^{n} \left(y_i - f_\theta(x_i)\right)^2, \qquad \theta = (\beta, w_1, \ldots, w_K),$$

but, because the weights are nested and the units symmetric, R **is not convex**: it has many local minima. Goodbye to the Lasso's guarantees.

Gradient descent: start at random weights θ^0 and take steps downhill,

$$\theta^{t+1} \;=\; \theta^t - \rho \nabla R(\theta^t),$$

with ρ the **learning rate**. In large networks the gradient is computed on mini-batches of data (*stochastic* descent; an **epoch** is one full pass; appendix A4).

- **Two strategies against overfitting** (ISL): learn slowly and stop in time, and penalize the weights. They are week 5's boosting strategies and week 3's Ridge.

Backpropagation: the residual, split up

ISL 10.7.1; the full derivation, in appendix A3

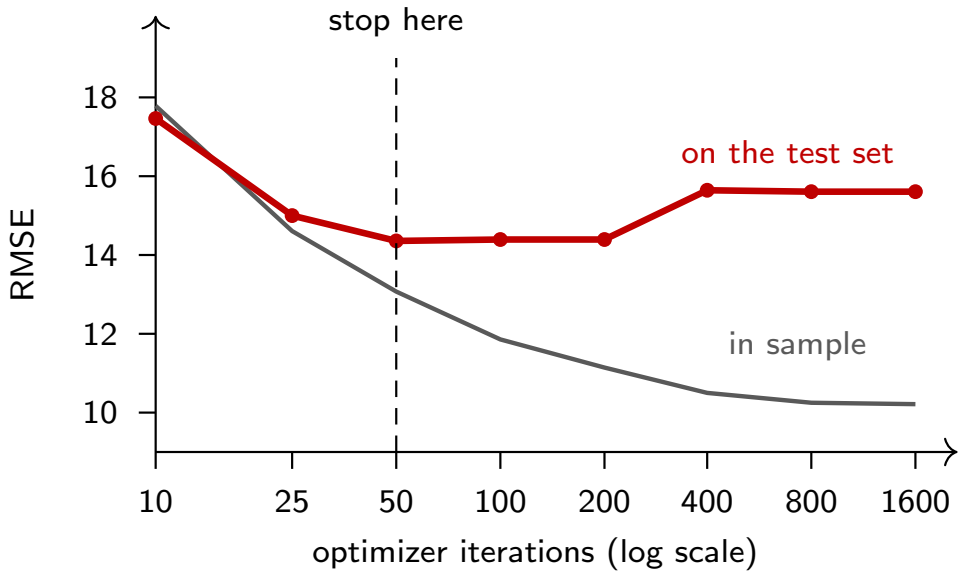
With $z_{ik} = w_{k0} + \sum_j w_{kj}\, x_{ij}$, the chain rule gives, for one observation,

$$\frac{\partial R_i}{\partial \beta_k} \;=\; -\big(y_i - f(x_i)\big)\, g(z_{ik}), \qquad \frac{\partial R_i}{\partial w_{kj}} \;=\; -\big(y_i - f(x_i)\big)\, \beta_k\, g'(z_{ik})\, x_{ij}.$$

- **Both formulas start from the residual** $y_i - f(x_i)$: differentiating splits the residual across the weights, first across the units by how much they activated, then across the inputs through each unit. Hence the name: the error travels backward from the output.
- It is boosting's residual again: there each tree fit the residual; here each step moves each weight in proportion to its share of the residual.
- Nobody derives it by hand: the packages do it themselves (automatic differentiation). The cost of the whole gradient is one forward pass and one backward pass.

Stopping in time and penalizing: the usual knobs

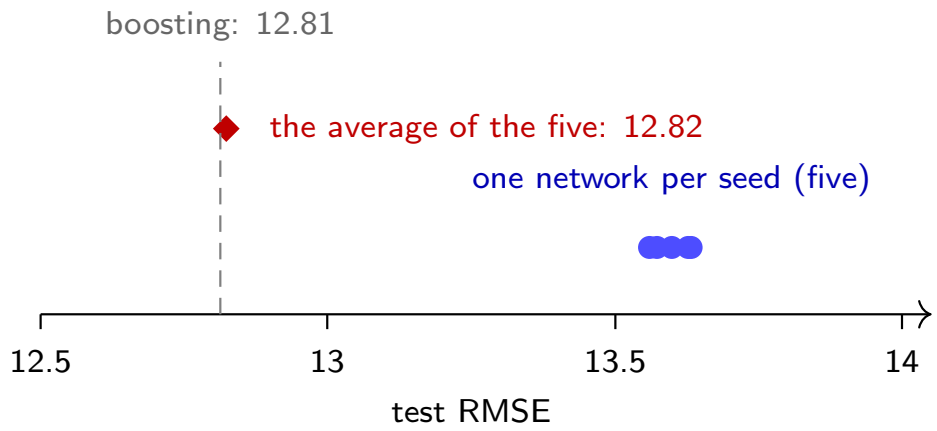
A 16-unit network with no weight decay, stopped after 10, 25, . . . , 1,600 iterations



- **In sample the error only falls** (from 17.8 to 10.2); on the test set it bottoms out at 50 iterations (14.3) and rises again (15.6). **Early stopping** is boosting's rounds knob; in practice you stop with a validation set, not with the test set.
- **The other knobs:** weight decay (Ridge on the weights), dropout (switching units off at random at each step: an implicit ensemble, like the forest), and, with images, augmenting the data with rotations and crops.

The seed is a hyperparameter too

The judge's chosen network, trained with five different seeds: each one's test error and their average's



- **Same data, same network, other initial weights:** the test error barely moves (from 13.56 to 13.63), but they are not the same function: since the problem is not convex, each seed ends up in a different local minimum.
- **Averaging the five networks** leaves 12.8: better than any one, and the same as boosting. It is the forest again: averaging models that err differently. Report the spread across seeds, never the best one.

The race: does the network beat the trees?

Test RMSE for the vote; weeks 3 and 5's, with the same split and the same test set

Ingredients	Linear	Network	Five networks	Boosting (S5)
The 37 census variables	OLS 19.8	16.9		15.7
Census + department	OLS 15.6	13.6	12.8	12.8
Census + department + coordinates	—	13,2		12,1

- **The network beats the linear models and loses to the trees** with all three ingredient sets. ISL 10.6 tells the same story with the *Hitters* salaries, and Grinsztajn, Oyallon, and Varoquaux (2022) document it on 45 tabular datasets: with n in the thousands and meaningful columns, tree ensembles rule.
- **Averaging five networks ties with boosting** (12.8), but added to week 5's average (boosting + k-NN: 11.4) they add nothing (11.5): their errors resemble boosting's (correlation 0.89).
- **That it does not win is a result.** The network pays off when there is **structure** the columns do not carry (images, text), when there are **millions** of data points, or to **borrow** a representation learned elsewhere.

1 Without y, in straight lines: principal components

2 Without y, in groups: k-means

3 With y: the neural network

4 How a network is fitted

5 When there are no columns: networks that see

6 Closing

Why a dense network cannot see

ISL 10.3: an image carries no meaningful columns; it carries pixels

A 224×224 color satellite photo is $224 \times 224 \times 3 = 150.528$ numbers.

- A first dense layer of 1,000 units would have about **150 million** weights: guaranteed overfitting with the thousands of examples an economist has.
- Worse: the dense network **ignores space**. To it, the corner pixel and its neighbor have no special relationship; if the roof shifts ten pixels, it is another input.

The two ideas that fix it (LeCun et al., 1998):

1. **Locality:** visual patterns are local; an edge lives in a small window.
2. **Shared weights:** an edge is an edge wherever it is, so the same window of weights sweeps the whole image.

Together they are called **convolution**, and they turn millions of weights into a few dozen per filter.

Convolution: a filter that sweeps the image

ISL 10.3.1. On the right, the 2022 vote as a grayscale image and the same map after an edge filter

Una imagen de 4×3 y un filtro de 2×2 (ISL, p. 413):

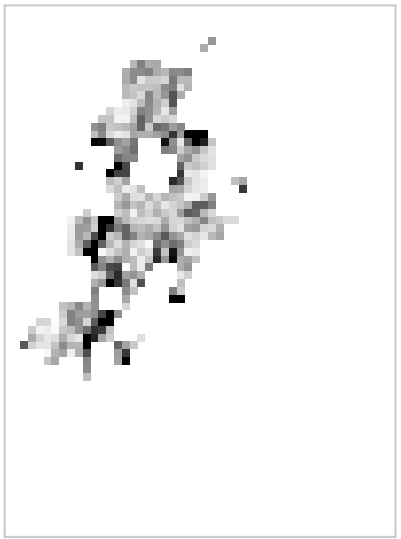
$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \\ j & k & l \end{bmatrix} * \begin{bmatrix} \alpha & \beta \\ \gamma & \delta \end{bmatrix}$$

the first cell of the output is $a\alpha + b\beta + d\gamma + e\delta$, and so on for every 2×2 window.

- Each output cell is a weighted sum of one window: high where the window **resembles** the filter.
- In a CNN the filter's weights are **learned**; here we use a fixed one (Sobel), which detects sharp changes.



the vote (darker, more Petro)



the edges (Sobel 3×3)

Each pixel is a quarter-degree cell; its shade, the Petro vote of its stations (937 cells with stations, the rest in white). The filter does not know what a department is: it finds **borders** because they are local changes.

Pooling, the architecture, and the borrowed network

ISL 10.3.2 a 10.3.5

Max pooling: summarize each 2×2 block with its maximum; the image halves along each axis and gains tolerance to small shifts.

$$\begin{bmatrix} 1 & 2 & 5 & 3 \\ 3 & 0 & 1 & 2 \\ 2 & 1 & 3 & 4 \\ 1 & 1 & 2 & 0 \end{bmatrix} \longrightarrow \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix}$$

The architecture: [convolution $\rightarrow$ ReLU $\rightarrow$ pooling] several times, and dense layers at the end. The first layers learn edges; the middle ones, textures and parts (roofs, roads, crops); the last ones, scenes.

Transfer learning: the borrowed network

A network trained on millions of generic images (ResNet50, ISL 10.3.5) already knows how to see edges and textures: it is **frozen** and its next-to-last layer is used as variables in a Ridge with your own data. **Jean et al. (2016)** fine-tune it to predict night lights from daytime photos and with those variables explain 37 to 55 % of consumption and 55 to 75 % of the asset index in African villages: that y is a PC1.

1. Without y, in straight lines: principal components
2. Without y, in groups: k-means
3. With y: the neural network
4. How a network is fitted
5. When there are no columns: networks that see
6. Closing

What these models say and what they do not

- **A principal component is a direction of variance,** not a concept: ours became a "deprivation index" when its loadings could be read and the vote audited it. And the one that varies most (PC1) is not the one that predicts most (PC2 explains 11 % of the vote; PC1, 0.9 %).
- **A k-means group is a convenient partition** until it survives another seed and can be named. Ours passed both tests; they describe better than they predict.
- **A network's units carry no coefficients to read:** to open it, week 6's kit works, outside the training data and with several seeds.
- **The table decides:** with the census variables, the department, and the coordinates, the network did not beat the boosting. Should we use deep learning here? No, and that is a good answer.

Y, not β, representations edition

The deprivation index orders surroundings; it does not explain the vote. That it shoots up in the most deprived decile does not say deprivation "causes" the vote: that decile is also the Pacific. Territory comes in bundles the census summarizes.

When to use what

	Use when	Watch out for
PCA	an index (assets, deprivation); looking at the data; compressing before a model with p near n	standardizing; the sign; linear only and variance only; nearly empty columns
k-means	segmenting to act (typologies, targeting)	scale; round groups; it always delivers K groups
Dense network	very large tabular data, or as a comparison	scale, seeds, decay; it rarely beats the trees
Borrowed CNN + Ridge	images with hundreds or thousands of labels	a domain far from the borrowed network's
Network from scratch	millions of examples and a GPU	all of the above, multiplied
Trees and boosting	mid-sized tabular data with meaningful columns	extrapolation; tuning

What to read: ISL 12.2 and 12.4 (PCA and k-means); ISL 10.1 to 10.3 and 10.6 to 10.7 (networks); Filmer and Pritchett (2001) for the asset index; Jean et al. (2016) for transfer learning with images; Grinsztajn et al. (2022) for the tabular evidence; Dell (2025) as the map of deep learning for economists.

Recap

1. **Without a y there is no test error:** stability, interpretation, and a y the method never saw stand in for it.
2. **PCA:** the directions of maximum variance, the best M-dimensional approximation. PC1 (32.0 %) is a deprivation index the vote audits (not in a straight line); compressing without y to predict does not beat OLS.
3. **k-means:** minimize the within-group variation; K by the elbow, the silhouette, or the use. Four nameable Colombias and two ruralities 21 points apart.
4. **A network** regresses y on variables it manufactures itself: directions chosen by the y and bent by g. With one unit it already beats PC1 (19.4 against 26.5).
5. **It is fitted** by going down the gradient (the residual, split up) and regularized with the usual knobs: early stopping, weight decay, averaging seeds.
6. **When:** in our table the network did not beat the trees; it pays off with images, text, millions of data points, or a borrowed representation.

Getting ready for next class...

To get the most out of next session

Tuesday, November 3 is the papers session. Six 15-minute presentations, one per field of economics, with the papers chosen in session 2. Script: the question and the decision; the data and the unit; the benchmark; how it is evaluated out of sample and where it fails; the main result; which method of the course it is and what we would have done differently.

Project Deliverable 1, the same day: the one-page idea (the question and the decision motivating it, the unit and the y, the data and what would be known at prediction time), sent before class, and a 5-minute pitch. Worth 10 of the project's 100 points.

Problem Set 2: due Tuesday, November 10 at 11:59 p.m. With today's session you have everything you need (Part 3 is the network against the forest); there is no class on November 10, but there are office hours.

Reading for the close: Athey and Imbens (2019), sections 1 to 3, for the causal bridge that ends session 8.

R application: the same stations: the census PCA and the U, the PCR with the judge, the four Colombias, the one-unit network, the units-and-decay grid, the seeds, and the race.

Essential references

- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2. ed.). Springer. Chs. 10 and 12. [ISL]
- Filmer, D., & Pritchett, L. (2001). Estimating Wealth Effects Without Expenditure Data—or Tears. *Demography*, 38(1), 115–132.
- Jean, N., Burke, M., Xie, M., Davis, W., Lobell, D., & Ermon, S. (2016). Combining Satellite Imagery and Machine Learning to Predict Poverty. *Science*, 353(6301), 790–794.
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? *NeurIPS Datasets and Benchmarks*.
- Dell, M. (2025). Deep Learning for Economists. *Journal of Economic Literature*, 63(1), 5–58.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521, 436–444.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chs. 6 to 9.
- Athey, S., & Imbens, G. (2019). Machine Learning Methods That Economists Should Know About. *Annual Review of Economics*, 11, 685–725.
- Gelvez, J. D., Cardiles, A., Martínez-González, E. F., & Muñoz, M. (2026). How Predictable Is an Election? Working paper. [The source of the case data.]

7 Appendix: going deeper

A1. PCA is an eigenvalue problem

Sea Σ la matriz de covarianza ($p \times p$) de las columnas centradas (y estandarizadas). La PC1 resuelve

$$\max_{\phi} \ \phi' \Sigma \phi \qquad \text{sujeto a} \qquad \phi' \phi = 1.$$

1. **Lagrangian:** $\mathcal{L}(\phi, \lambda) = \phi' \Sigma \phi - \lambda (\phi' \phi - 1)$.
2. **First-order condition:** $2\Sigma\phi - 2\lambda\phi = 0 \implies \boxed{\Sigma \phi = \lambda \phi}$: ϕ is an **eigenvector** of Σ.
3. **Which one?** The projection's variance is $\phi' \Sigma \phi = \lambda \, \phi' \phi = \lambda$: PC1 is the eigenvector of the **largest** eigenvalue; PC2, of the second (orthogonality comes free because Σ is symmetric), and so on. Component m's PVE is $\lambda_m / \sum_k \lambda_k$.
4. **In practice,** `prcomp` uses the singular value decomposition of the centered matrix, $X = UDV'$: the columns of V are the loadings, UD the scores, and d_m^2/n the variances. Truncating the sum at M terms gives the best rank-M approximation (Eckart and Young, 1936): the main body's "other face".

A2. Hierarchical clustering and Gaussian mixtures

- **Why each k-means step lowers the objective:** in step (a), the mean is the constant minimizing the within-group sum of squares; in (b), moving a station to the nearest centroid can only reduce its distance. The objective never rises and partitions are finite: the algorithm terminates.
- **Agglomerative hierarchical** (ISL 12.4.2): start with n one-station groups and merge the two closest at each step. The **dendrogram** shows every resolution at once; cutting it at a height is choosing K after seeing the structure. "Closest" is defined by the linkage (complete, average, Ward). It costs on the order of n^2 distances.
- **Gaussian mixtures:** the data come from K normals with weights π_k; each station receives a **probability** of belonging to each group (the EM algorithm). It allows elongated, differently sized groups; k-means is the limiting case of equal variances going to zero.
- **Nonlinear maps** (t-SNE, UMAP): they serve to *look at* high-dimensional data; global distances and group sizes on the map mean nothing.

A3. Backpropagation: the one-hidden-layer derivation

The model (regression, one observation's squared loss):

$$z_k = w_{k0} + \sum_j w_{kj}\, x_j, \quad A_k = g(z_k), \quad f = \beta_0 + \sum_k \beta_k A_k, \quad R = \tfrac{1}{2}\,(y - f)^2.$$

1. The output layer: $\dfrac{\partial R}{\partial \beta_k} = -(y - f)\, A_k$: the residual, split by how much each unit activated.

2. The hidden layer, by the chain rule through f and g:

$$\frac{\partial R}{\partial w_{kj}} = \frac{\partial R}{\partial f} \cdot \frac{\partial f}{\partial A_k} \cdot \frac{\partial A_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{kj}} = -(y - f)\, \beta_k\, g'(z_k)\, x_j.$$

3. Backward: unit k's "error" is $\delta_k = -(y - f)\, \beta_k\, g'(z_k)$, and all its gradients are $\delta_k\, x_j$. With more layers, the same step repeats layer by layer, from the output toward the input, reusing the δ's already computed.

With ReLU, $g'(z) = \mathbb{I}\{z > 0\}$: the error only flows through active units. With the sigmoid, $g'(z) = g(z)(1 - g(z))$, nearly zero in the tails: the gradient vanishes.

A4. Stochastic gradient descent, momentum, and Adam

Mini-batches: instead of summing the gradient over the n data points, a batch of 32 to 256 is used at each step; an **epoch** is one full pass. Noisy but cheap, and the noise helps escape bad valleys.
Momentum: move in the direction of a moving average of the gradient,

$$m_t = \beta_1\, m_{t-1} + (1-\beta_1)\, g_t, \qquad \theta_t = \theta_{t-1} - \rho\, m_t.$$

Adam (Kingma and Ba, 2015): it additionally scales each parameter's step by its own volatility,

$$v_t = \beta_2\, v_{t-1} + (1-\beta_2)\, g_t^2, \qquad \theta_t = \theta_{t-1} - \rho\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

with $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$. The usual values: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\rho \approx 10^{-3}$.

- `nnet`, the application's package, uses no batches: it optimizes with BFGS (a quasi-Newton) over all the data, the sensible choice for networks of hundreds of weights. Million-weight networks use Adam.

A5. The arithmetic of a convolutional layer

Output size. An $n \times n$ image, a $k \times k$ filter, padding p, and stride s produce a map of side

$$\left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

With 224×224, $k = 3$, $p = 1$, and $s = 1$ the output is still 224×224; a 2×2 pooling brings it down to 112×112.

Parameters. A layer with F filters of $k \times k$ over a C-channel input has $F\,(k^2 C + 1)$ weights, *regardless of the image's size*. Sixty-four 3×3 filters on a color photo: $64 \times (27 + 1) = 1.792$ weights. The equivalent dense layer on $224 \times 224 \times 3$ inputs and 64 outputs would have about 9.6 million.

ResNet50: about 25 million parameters and 50 layers with "shortcuts" that add each block's input to its output, so the gradient does not vanish. Its next-to-last layer has 2,048 numbers per image: the transfer-learning variables.

A6. Universal approximation: what it says and what it does not

The theorem (Cybenko, 1989; Hornik, 1991). If g is not a polynomial (the sigmoid, ReLU), for every continuous f on a compact set and every $\varepsilon > 0$ there exist K and weights such that

$$\sup_x \left| f(x) - \sum_{k=1}^{K} \beta_k \, g\left(w_k' x + b_k\right) \right| < \varepsilon.$$

- **It is a density result,** like Weierstrass's for polynomials. It does not say how many units are needed (K can grow exponentially with p), nor that gradient descent finds them, nor anything about the test error.
- **Depth:** there are functions a deep network represents with few units while a one-layer one needs exponentially many (Telgarsky, 2016): composition reuses pieces.
- **The week 2 analogy:** splines also approximate any smooth function with fixed knots; the network learns *where* to put the knots and on which combinations of variables.

Appendix references and others cited

- Eckart, C., & Young, G. (1936). The Approximation of One Matrix by Another of Lower Rank. *Psychometrika*, 1(3), 211–218. [A1]
- Rousseeuw, P. J. (1987). Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15, 1929–1958.
- Kingma, D., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*. [A4]
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR*. [A5]
- Cybenko, G. (1989). Approximation by Superpositions of a Sigmoidal Function. *Mathematics of Control, Signals and Systems*, 2, 303–314. [A6]
- Hornik, K. (1991). Approximation Capabilities of Multilayer Feedforward Networks. *Neural Networks*, 4(2), 251–257. [A6]
- Telgarsky, M. (2016). Benefits of Depth in Neural Networks. *COLT*. [A6]